
Workgroup:	HTTP Working Group (httpbis)
Internet-Draft:	draft-prakash-http-subscribe-00
Published:	4 July 2026
Intended Status:	Standards Track
Expires:	5 January 2027
Author:	A. Prakash <i>Boeing</i>

The HTTP SUBSCRIBE Method

Abstract

This document defines the HTTP SUBSCRIBE request method. The SUBSCRIBE method allows a client to establish a long-lived, safe connection to a resource to receive real-time updates and event streams. It enables servers to push data to clients using standard HTTP structures (such as HTTP/2 or HTTP/3 streams) while supporting a request body for subscription parameters, avoiding the protocol-switching overhead of WebSockets and the URL limitations of Server-Sent Events (SSE) via GET.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 5 January 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	2
2. Terminology	3
3. The SUBSCRIBE Method	3
3.1. Request Semantics	3
3.2. Response Semantics	4
4. Client and Server Behavior	4
4.1. Client Behavior	4
4.2. Server Behavior	5
5. Caching Considerations	5
6. Security Considerations	5
6.1. Connection Exhaustion (DoS)	5
6.2. Authentication and Authorization	5
7. IANA Considerations	6
8. References	6
8.1. Normative References	6
8.2. Informative References	6
Author's Address	6

1. Introduction

Modern web applications require efficient, low-latency, and real-time communication channels from the server to the client. Examples include chat applications, stock market tickers, collaborative document editing, and live dashboards.

Historically, real-time push has been achieved using three primary workarounds:

1. **Long Polling:** Recreating HTTP requests continuously. This is highly inefficient and creates substantial connection overhead.

2. **WebSockets:** Upgrades the connection from HTTP to a separate bidirectional TCP-based protocol. While efficient, it bypasses HTTP intermediaries (like load balancers, reverse proxies, and Web Application Firewalls), breaks semantic caching, complicates authentication, and often struggles with strict enterprise firewalls.
3. **Server-Sent Events (SSE):** Utilizes standard HTTP GET requests with `text/event-stream` responses. However, because GET requests do not support a request body, clients must pass subscription parameters (such as filter expressions, selected fields, or authentication tokens) within the URL query string. This leads to problems with URL length limits, logging of sensitive data, and overall architectural rigidity.

This specification introduces the SUBSCRIBE method to address these issues. SUBSCRIBE is a safe HTTP method that allows the client to send subscription parameters in the request body while establishing a long-lived, server-push event stream.

2. Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

3. The SUBSCRIBE Method

The SUBSCRIBE method is used to request a persistent, real-time event stream from a target resource.

- **Safe:** Yes. A SUBSCRIBE request is read-only; it MUST NOT modify the state of the target resource on the server.
- **Idempotent:** Yes. Multiple identical SUBSCRIBE requests will yield identical event stream configurations.
- **Request Body:** Allowed. The request body contains subscription parameters (e.g., query filters, requested fields, backfill start time, or sub-topics).
- **Response Body:** Allowed. The response body is a long-lived stream of events or data updates.

3.1. Request Semantics

Clients send a SUBSCRIBE request to the target URI. The request body SHOULD specify the parameters of the subscription. For example, a JSON payload may describe a topic filter:

```
SUBSCRIBE /events/trades HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: text/event-stream

{
  "ticker": "GOOG",
  "min_volume": 100,
  "fields": ["price", "volume", "timestamp"]
}
```

3.2. Response Semantics

A successful response is indicated by the 200 OK status code. The response body **MUST** consist of a continuous stream of structured data chunks.

To maintain the subscription, the server holds the response stream open indefinitely.

- **HTTP/1.1:** The server **MUST** use chunked transfer encoding (`Transfer-Encoding: chunked`) to stream data.
- **HTTP/2 and HTTP/3:** The server streams data over a single multiplexed stream, utilizing native frame transport without requiring chunked transfer encoding.

The response `Content-Type` **SHOULD** indicate a streaming protocol, such as `text/event-stream` or `application/x-ndjson`.

Example response header and initial stream chunk:

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache, no-store
Connection: keep-alive

data: {"price": 175.20, "volume": 150, "timestamp": 1719999999}
data: {"price": 175.25, "volume": 500, "timestamp": 1720000005}
```

4. Client and Server Behavior

4.1. Client Behavior

A client initiates a subscription by issuing a `SUBSCRIBE` request. The client **MUST** be prepared to read the response body incrementally as data chunks arrive.

If the client wishes to terminate the subscription, it **MUST** close the transport-level stream (or connection).

4.2. Server Behavior

Upon receiving a SUBSCRIBE request, the server:

1. MUST parse and validate the request body and headers.
2. MUST verify authorization for the requested subscription.
3. On success, MUST send response headers (e.g., 200 OK) and keep the stream active.
4. MUST push events to the stream as they occur.
5. SHOULD periodically transmit keep-alive/heartbeat chunks (such as empty comments in `text/event-stream`) to prevent intermediary timeouts.

If the request is invalid or unauthorized, the server MUST return an appropriate 4xx or 5xx status code and close the connection immediately.

5. Caching Considerations

Because SUBSCRIBE responses represent dynamic, real-time streams of events, they MUST NOT be cached by shared caches or HTTP intermediaries.

Servers MUST include a `Cache-Control: no-cache, no-store` header in the response. Intermediaries MUST immediately forward both the request and response stream without buffering.

6. Security Considerations

6.1. Connection Exhaustion (DoS)

Long-lived connections consume file descriptors and memory. Malicious clients could open thousands of subscriptions to deplete server resources (Denial of Service).

Servers SHOULD:

- Enforce limits on the number of concurrent subscriptions per client/IP address.
- Support HTTP/2 and HTTP/3 to multiplex subscriptions over a minimal number of TCP/QUIC connections.
- Implement aggressive timeouts for clients that do not read stream chunks.

6.2. Authentication and Authorization

Subscriptions often contain sensitive live data. Because connections are long-lived, token expiration (e.g., OAuth tokens) must be handled. Servers SHOULD validate token longevity during the initial request handshake. If a token expires while a stream is active, the server MAY push an expiration event and close the connection, forcing the client to re-authenticate and re-subscribe.

7. IANA Considerations

IANA is requested to register the SUBSCRIBE method in the "HTTP Method Registry" under the Hypertext Transfer Protocol (HTTP) Parameters registry:

- **Method Name:** SUBSCRIBE
- **Safe:** Yes
- **Idempotent:** Yes
- **Reference:** This document

8. References

8.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <<https://www.rfc-editor.org/info/rfc9110>>.

8.2. Informative References

- [RFC5789] Dusseault, L. and J. Snell, "PATCH Method for HTTP", RFC 5789, DOI 10.17487/RFC5789, March 2010, <<https://www.rfc-editor.org/info/rfc5789>>.
- [RFC10008] IETF, "The HTTP QUERY Method", June 2026.

Author's Address

Amit Prakash

Boeing

Email: amitbcet2k15@gmail.com